Ders 7 Çalışma Özeti

Ders 7 Çalışma Özeti

Genel Konular

- Template Method kalıbı: amaç ve sorun (ekrandan, 00:05:20)
  - Kalıtımın getirdiği çok biçimliliği ve metot override etmeyi disipline eden bir kalıptır. Amaç: bir algoritmanın sabit kalması gereken ana adımlarını belirleyip değişebilecek ara adımların ayrıntılarının belirlenmesini ertelemek. Ana adımlar üst sınıfta, ara ayrıntılar alt sınıflarda yazılır.
  - Slayttaki örnek: farklı belge türleriyle çalışabilecek uygulamalar geliştirmeyi sağlayan bir çerçeve program (framework). Kullanıcı, çerçevenin sunduğu Belge ve Uygulama sınıflarından kalıtımla yeni sınıflar türeterek kendi yazılımını hazırlar.
  - Sorun: belge açarken ya da oluştururken her türlü ince ayrıntıyı kullanıcıya atmak istemiyoruz; bazı temel adımların uygun sırayla yürütülmesini çerçeve programa bırakmak istiyoruz.
- Template Method GoF örnek çözümü (ekrandan, 00:10:00)
  - Hoca şemanın ortasından yatay bir çizgi çizdi: çizginin üstü framework (soyut Document ve Application), altı bizim yazacağımız uygulama (MyDocument, MyApplication).
  - Document: Save(), Open(), Close() somut; DoRead() soyut (italik). Open() bir dosya tutamacı alır, okunan baytların nasıl yorumlanacağı ise belge türüne bağlı olduğundan DoRead()'e bırakılır.
  - Application: AddDocument(), OpenDocument() somut; DoCreateDocument(), CanOpenDocument(), AboutToOpenDocument() soyut. MyApplication.DoCreateDocument() `return new MyDocument` yapar.
  - Template metot OpenDocument'tır (ekrandan, 00:10:00):

```
void Application::OpenDocument (const char* name) {
    if (!CanOpenDocument(name)) {
        // cannot handle this document
        return;
    }
    Document* doc = DoCreateDocument();
    if (doc) {
        _docs->AddDocument(doc);
        AboutToOpenDocument(doc);
        doc->Open();
        doc->DoRead();
    }
}
```

  - CanOpenDocument'ta denetlenen şey dosya erişimi değil, belgenin biçiminin uygun olup olmadığı, verinin tam ya da bozuk olup olmadığıdır.
- Kalıtımdan farkı ve final
  - Burada tek bir sınıftan kalıtım değil, birbiriyle ilişkili iki ayrı kalıtım hiyerarşisi söz konusudur.
  - Soyut üst sınıftaki somut template metot final tanımlanmalıdır; yoksa bir alt sınıf OpenDocument'ı override ederek bütün framework'ün çalışma biçimini bozabilir.
- Template Method örnek gerçeklemesi: farklı veritabanlarından kayıt çekme (ekrandan, 00:17:10, 00:18:30)
  - Kaynak: Design Patterns Explained. Uygulama SQL ile kayıt çekecek ama farklı ve-

ritabanı yönetim sistemleriyle çalışabilmeli. SQL standart olsa da bağlantı kurma biçimi ve özellikle stored procedure desteği sistemden sisteme değişir.
  - Değişmeyen adımlar: (1) bağlantı dizesi (connection string) oluştur, (2) bağlantı komutunu gönder, (3) select komutunu inşa et, (4) komutu gönder, (5) oluşan veri kümesini döndür.
  - Sınıf şeması: QueryControl (`- qt : QueryTemplate`, `+ QueryControl(QueryTemplate)`) bir «abstract» QueryTemplate'e bağlıdır. QueryTemplate: `+ doQuery(DBname, querySpec : String) {final}`, `~ formatConnect(DBname : String) {abstract}`, `~ formatSelect(querySpec : String) {abstract}`. Alt sınıflar: OracleQT, SQLServerQT, GraphDBQT.
  - Slayt notu: komut biçimlendirme metotları iç çalışmayla ilgili görüldüğünden protected olabilir; bir sınıf kütüphanesi ya da çerçeve program hazırlanmadığı için Java'da paket görünürlüğü kullanılabilir. Hoca, böylece bu metotların başka paketten çağrılamayacağını söyledi.
  - İskelet kod (ekrandan, 00:18:30):

```
package templateMethod.example;
public abstract class QueryTemplate {
    public final void doQuery( String DBname,
                               String querySpec ) {
        String dbCommand;
        dbCommand = formatConnect( DBname );
        //dbCommand komutunu yürüterek bağlantıyı kur.
        dbCommand = formatSelect( querySpec );
        //dbCommand komutunu yürüterek kayıtları oku.
        //Oluşan veri kümesini geri döndür.
    }
    abstract String formatConnect( String DBname );
    abstract String formatSelect( String querySpec );
}
```

- Template Method ile Strategy karşılaştırması
  - İlk akla Strategy gelebilir; ama Strategy'nin üst türü genelde bir arayüzdür ve her somut strateji işin tamamını yapar. Soyut ya da somut üst sınıf yalnız bir varsayılan algoritmanın işe yarayacağı nadir durumda düşünülür.
  - Template Method'da bazı ayrıntılar sabit, bazıları alt sınıfta değişir. Veritabanı örneğinde Strategy problemin yarısına hitap eder; ortak yapılacak kısımları görüp Template Method'u seçmek daha doğrudur.
- Visitor kalıbı: amaç, sorun ve eleştiri (ekrandan, 00:24:10)
  - Amaç: herhangi bir nesne yapısını (çoğunlukla bir Composite ya da başka bir ağaç; liste de olabilir) oluşturan elemanlar üzerinde yapılacak işlemler tanımlamak.
  - Sorun: nesne yapısı her türden bağıntı içeren sınıfların örnekleri arasında kurulmuş olabilir; üzerinde işlem yapılan elemanların sınıf yapılarını değiştirmek istemiyoruz. Hoca, kaynak kodun kapalı olduğu bir durum olmadığını, projeyi sıfırdan yazarken ileride gelecek yeni ihtiyaçları parça-bütün yapısındaki sınıflara ekleme yapmadan karşılamanın hedeflendiğini vurguladı.
  - Eleştiri (slayttan): madem yapılacak işlemler var, eylemler ve sorumluluklar tam belirlenmemiş demektir; mevcut sınıf yapılarını değiştirerek farklı bir tasarım yapmak daha doğru olabilir. Hoca bu hedefi biraz "haddini aşan" (ambitious) buldu.
  - Ne zaman mantıklı: geriye doğru uyumluluğu çok uzun süre korumak istenen bir framework'te, zaman içinde öngörülemeyen birçok ek işlev istenebilecekse.
- Visitor GoF örneği: derleyici (00:24:01, ekrandan, 00:30:00)
  - Derleyici kaynak kodu farklı türden token'lara (Node) ayırır ve bunlar üzerinde çeşitli işlemler yapar; farklı tip parçalarda işlemler farklı yürütülür.
  - Hoca `int x = 3+5` komutunun ağacını çizdi: int tür düğümü, x değişken düğümü, atama düğümü, toplama düğümü ve onun iki işleneni 3 ve 5 sabit düğümleri.
  - Bu ağaç üzerinde yapılabilecek işler: pretty printing (okunabilirlik için boşluk ekleme), vurgulama (yalnız tür anahtar kelimesini koyu yapma; diğer düğümler atlanır), hata ayıklamada yalnız değişken düğümlerini dolaşıp x'in değerini izleme, atamada tür denetimi (type checking), kod üretimi (code generation).
  - Kalıpsız çözümde her işlem Node hiyerarşisinde bir metottur; o işin anlamsız olduğu düğümlerde de (boş gövdeli bile olsa) kodlanmak zorundadır.

  - Visitor çözümü (slayt 88): NodeVisitor (VisitAssignment(AssignmentNode), VisitVariableRef(VariableRefNode)); TypeCheckingVisitor ve CodeGeneratingVisitor. Program–Node parça-bütün ilişkisi; Node'a `Accept(NodeVisitor)` eklenir; AssignmentNode.Accept(v) içinde `v->VisitAssignment(this)`, VariableRefNode.Accept(v) içinde `v->VisitVariableRef(this)` çağrılır.
  - Düğüm "ben bir assignment düğümüyüm, beni assignment olarak ziyaret et" der; ziyaretçi "daveti icabet eder" ve yalnız o düğümde yapılması gereken işi yapar.
  - Zayıf yön (slayttan): düğümler ziyaretçinin hangi metodunun kendi türleri için olduğunu bilmelidir; bu ek bir bağlaşımdır.
- Visitor kalıp yapısı ve zayıf yönleri (ekrandan, 00:36:50, 01:50:00)
  - Bileşenler: ObjectStructure (elemanları dolaşılacak yapı); Element (ortak arayüz, Accept metodu burada tanımlanır); ConcreteElementX (ziyaretçinin hangi metodunun kendi türü için olduğunu bilir); Visitor (her tür eleman için ziyaret işlemlerinin tamamı burada tanımlanmak zorundadır); ConcreteVisitorX (elemanlar ayrı ayrı ziyaret edildiğinden ziyaretler arasında durum bilgisini saklayacak kodlama yapılmalıdır).
  - Yeni bir iş türü eklemek kolaydır (yeni somut ziyaretçi). Yeni bir ConcreteElement türü eklemek ise zordur: Visitor arayüzüne VisitConcreteElementC eklenir ve o elemanla işi olmayan ziyaretçiler bile bu metodu boş gövdeyle kodlamak zorunda kalır.
  - Sonuç: parça-bütün yapısının parçaları oturmuşsa ve değişen yalnız yapılacak işlerse Visitor uygulanır.
- Visitor örnek gerçeklemesi: proje yönetim yazılımı, Çözüm 1 (ekrandan, 00:40:00, 00:46:00, 01:00:40)
  - Kaynak: Applied Java Patterns. Varlıklar: Project (hiyerarşinin kökü), Task (projede tanımlı görev), Deliverable (projenin ara çıktısı veya sonucu olarak üretilecek kod, belge, ürün vb.).
  - «Abstract» ProjectItem (name, description : String ve getter/setter'lar); alt sınıflar Deliverable (materialsCost, productionCost : double), Task (projectItems : ArrayList, timeRequired : double, add/removeProjectItem), Project (projectItems, add/removeProjectItem). Task ve Deliverable'ın birer sahibi (owner) Contact vardır.
  - Hoca, Project tarafında bir Composite yapısı olduğunu söyledi. Task da alt görev ve deliverable içerebiliyorsa tam bir Composite olur; bu durumda çevrimsel bağımlılığa (bir görevin altına projenin kendisini koymak) dikkat edilmelidir, sonsuz döngü oluşabilir.
  - Şemada Task'tan ProjectItem'a parça-bütün ilişkisi (elmas) eksikti; hoca bunu çizdi. Task yalnız deliverable içerecekse ilişki Deliverable'a kurulmalıdır; iki elmastan biri seçilir.
  - ProjectCalculator: `calculateTimeRequired()`, `calculateCost()` public; alt görevleri de hesaplayan `calculateTimeRequired(subTask : Task)` ve `calculateCost(subTask : Task)` private. Report sınıfı main içerir.
  - Kodda her düğümde `if( item instanceof Task )` ile tür denetimi ve `(Task)item` dönüşümü yapılır (ekrandan, 01:00:40). Hoca bunun nesneye dayalılığın iyi bir kullanımı olmadığını, kodun esnekliğini azalttığını söyledi.
  - Örnek veri: bir yüksek lisans eğitimi proje olarak modellenmiştir; görevler dönemler, deliverable'lar dersler ve tez metnidir. Deliverable kurucusunun son iki sayısı derse çalışma ve sınava hazırlık saatleri olarak kullanılmıştır (örneğin 3 saat × 15 hafta = 45). Program çıktısı: "Proje süresi: 60.0 hafta.", "Proje maliyeti: 1174.0 saat çalışma." (ekrandan, 01:10:00).
- Visitor örnek gerçeklemesi: Çözüm 2 (ekrandan, 00:50:00, 01:12:30, 01:14:40)
  - ProjectItem'a `+ accept(ProjectVisitor) {abstract}` eklenir; Deliverable, Task ve Project bunu somut olarak kodlar. «interface» ProjectVisitor: visitDependentTask(DependentTask), visitDeliverable(Deliverable), visitTask(Task), visitProject(Project). Gerçeklemeler: ProjectCostVisitor ve ProjectTimeVisitor; Report ikisini de kullanır.
  - Hoca, örneği sadeleştirmek için DependentTask sınıfını şemaya koymadığını, bu yüzden visitDependentTask'ın da şemadan silinmesi gerektiğini not aldı.
  - ProjectCostVisitor'ın kurucusu `project.accept(this);` ile dolaşmayı kökten başlatır; `visitDependentTask(p)` yalnız `visitTask(p)` çağırır. Report'ta `new ProjectCostVisitor(rep.project)` ile `vc.getCost()`, `new ProjectTimeVisitor(rep.project)` ile `vt.getTime()` yazdırılır.

  - Ziyaretçilerde hiç instanceof yoktur; hangi düğümün ziyaret edildiği metot parametresinin türüyle bildirilir. Çıktı yine 60 hafta ve 1174 saattir.
  - Sorumluluk ayrımı: maliyet ve süre hesapları artık ayrı sınıflardadır. Yinelenen kod da biraz azalmıştır; ziyaretçilerdeki benzerlik tek satırlık döngülerden ibarettir.
- Mediator kalıbı (ekrandan, 01:33:15, 01:34:00, 01:38:30, 01:40:00)
  - Amaç: bir nesneler kümesinin aralarındaki etkileşimleri sarmalamak. Nesneler birbiriyle doğrudan değil dolaylı ilişki kurar; böylece aralarındaki ilişkiler nesneler değiştirilmeden değiştirilebilir.
  - GoF örneği: font seçme penceresi (Font Chooser). Liste kutusu, giriş alanı, radyo düğmeleri gibi bileşenlerde yapılan her değişiklik üstteki örnek metin alanına yansıtılır. Radyo düğmeleri kendi aralarında da ilişkilidir (birine tıklanınca diğeri boşalır). Nesne şemasında aClient, aListBox, aButton, anEntryField hep aFontDialogDirector'a bağlıdır.
  - Örnek çözüm (slayt 109): soyut DialogDirector (ShowDialog(), soyut CreateWidgets(), soyut WidgetChanged(Widget)); FontDialogDirector bunları gerçekler ve ListBox'a (list) ve EntryField'a (field) bağlıdır. Widget (Changed()) her bileşenin üst sınıfıdır; Changed() içinde `director->WidgetChanged(this)` çağrılır. ListBox: GetSelection(); EntryField: SetText(). Slayta göre soyut sınıftan gerçeklemeler oluşturmak için Template Method kullanılır.
  - Widget hiyerarşisi GUI framework'ünde zaten vardır; olay yakalama da framework'te Observer türü listener'larla ya da Command yapısıyla zaten kuruludur. Bize düşen, change metodunu (yoksa) eklemek ve bileşenleri birbirine değil director'a bağlamaktır.
  - Kalıp yapısı (slayt 110): Colleague'lar birbirleriyle doğrudan değil bir Mediator üzerinden konuşur. Mediator–Colleague arasında `mediator` ilişkisi; ConcreteMediator, ConcreteColleague1 ve ConcreteColleague2'yi tanır.
  - Kalıbın önerisi: N^2 etkileşimi araya bir $(N+1)$'inci bileşen koyarak N etkileşime indirmek. Üst sınıfa varsayılan davranışlar eklenebilir; kalıp kalıtımın her olanağına açıktır.
  - Eleştiri: sınıf düzeyinde bağlaşım değişmez, yalnız ilişkideki nesne sayısı azalır. Karmaşıklık Colleague'lardan ConcreteMediator'a taşınır; Mediator çoğu zaman Large Class kod kusuruna sahip olur. Etkileşimler karmaşıksa yarar sağlar; değilse değmeyebilir.
- UML gösterimi üzerine hatırlatmalar (ekrandan, 01:50:00)
  - Kalıtım oku standart UML'e göre çizilmelidir; Dörtlü Çete'nin kitabı UML'den önce çıktığı için eski gösterimi kullanır.
  - Soyut sınıf el yazısında italikle anlaşılmaz; `<<abstract>>` stereotipi ya da arayüz için `<<interface>>` yazılır. Metotların dönüş türü iki noktadan sonra yazılır; soyut ve final metotlar `{abstract}`, `{final}` ile işaretlenir. Template Method'da template metodun final işaretlenmesi kalıbın önemli bir parçasıdır.

Hocanın Özellikle Vurguladığı Kısımlar

- Template metot final olmalı
  - Soyut üst sınıftaki somut template metot final tanımlanmazsa bir alt sınıf onu override edip framework'ün çalışma biçimini bozabilir (00:10:51). UML'de de `{final}` olarak işaretlenmesi beklenir (01:49:18).
- Doğru kalıbı seçmek ve gerekçesini anlatmak
  - Veritabanı örneğinde Strategy problemin yarısını çözer; ortak adımları görüp Template Method'u seçmek gerekir (00:15:14). Benzer görünen kalıplar arasında problemin bütününe hitap edeni seçmek önemlidir.
- Visitor'un eleştirisi ve ne zaman kullanılacağı
  - Yapılacak işler belli değilse sorumluluk ataması eksik olabilir; Visitor ancak parça-bütün yapısı oturmuşsa ve uzun süre geriye doğru uyumluluk gerekiyorsa mantıklıdır (00:22:38, 00:23:31, 00:38:06).
- instanceof kullanmak kötü bir işarettir
  - "Bu nesne şu türden mi, öyleyse dönüştür ve şunu yap" diye kod yazmak nesneye dayalılığın iyi bir kullanımı değildir; Visitor çözümü bunu ortadan kaldırır (01:02:13, 01:15:16).

- Kalite hedefleri araçtır, amaç değil
  - Bir metriği iyileştirmek için harcanan çaba süreçte başka verimsizlikler doğurabilir (01:19:19).
- Mediator karmaşıklığı yok etmez, taşır
  - Karmaşıklık ConcreteMediator'da toplanır; Mediator Large Class kusuruna sahip olabilir (01:43:19).
- UML kurallarına uyma
  - GoF'un eski gösterimi yerine standart UML kalıtım oku; italik yerine `<<abstract>>`; `{abstract}` ve `{final}` işaretleri (01:47:16–01:49:45).

Kısa Tekrar Notları

- Template Method: algoritmanın iskeleti üst sınıftaki final bir metotta, değişen adımlar soyut metotlarda; alt sınıflar yalnız bu adımları doldurur.
- GoF örneği: Application.OpenDocument template metottur; CanOpenDocument, DoCreateDocument, AboutToOpenDocument ve Document.DoRead alt sınıfa bırakılır.
- Veritabanı örneği: QueryTemplate.doQuery {final}; formatConnect ve formatSelect {abstract}, paket görünürlüğünde; OracleQT, SQLServerQT, GraphDBQT.
- Strategy: her somut strateji işin tamamını yapar. Template Method: ortak ana adımlar üst sınıfta.
- Visitor: elemanlara `accept(Visitor)` eklenir; eleman kendi türüne uygun `visitX(this)` metodunu çağırır.
- Visitor ile yeni işlem eklemek kolay, yeni eleman türü eklemek zordur.
- Proje yönetimi örneği: ProjectItem, Project, Task, Deliverable, Contact; Çözüm 1'de ProjectCalculator instanceof kullanır, Çözüm 2'de ProjectCostVisitor ve ProjectTimeVisitor kullanılır. Her iki çözüm de 60 hafta ve 1174 saat verir.
- Mediator: Colleague'lar birbirine değil Mediator'a bağlıdır; N^2 etkileşim yerine N etkileşim.
- Mediator'un riski: ConcreteMediator Large Class'a dönüşebilir.
- UML: `<<abstract>>`, `<<interface>>`, `{abstract}`, `{final}`, dönüş türü iki noktadan sonra.

Detaylı Açıklamalar

- Template Method'un özü, hangi işin hangi sırayla yapılacağının üst sınıfta sabitlenmesidir. GoF'un belge örneğinde framework, bir belgenin açılıp açılamayacağını sorma, belgeyi oluşturma, listeye ekleme, açmadan önce yapılacakları yapma, açma ve okuma adımlarının sırasını OpenDocument içinde kodlar. Bu adımlardan belge türüne bağlı olanlar soyut metot olarak bırakılır ve kullanıcı yalnız onların gövdesini yazar. Hoca, kalıbın "kalıtımdan farkı ne" sorusuna iki cevap verdi: birincisi, tek bir sınıf değil birbiriyle ilişkili iki kalıtım hiyerarşisi (Document ve Application) birlikte tasarlanır; ikincisi, template metodun final olması gerekir. Final olmazsa kalıp bir şablon olmaktan çıkar.
- Veritabanı örneğinde beş adım her veritabanı için aynıdır; yalnız bağlantı ve select komutunun biçimlendirilmesi değişir. QueryControl, bir QueryTemplate nesnesi alır ve onun doQuery metodunu çağırır. Somut alt sınıflar (Oracle, SQL Server, hatta bir graf veritabanı için) yalnız formatConnect ve formatSelect metotlarını yazar. Slayt, bu metotların protected olabileceğini ama bir kütüphane hazırlanmadığından paket görünürlüğünün de yeteceğini söyler; hoca paket görünürlüğünü, metotların başka paketten çağrılıp çalışma bütünlüğünün bozulmasını önlemek için seçtiğini anlattı. Hoca bu kalıbı "çok zor değil ama akılda tutulması yararlı bir çözüm" diye niteledi.
- Template Method ile Strategy'nin farkı veritabanı örneği üzerinden konuşuldu: farklı veritabanlarını desteklemek Strategy'yi akla getirir, ama Strategy'nin her somut stratejisi işin tamamını yapar ve üst türü genelde bir arayüzdür. Bu problemde işin ana hatları sabittir; o yüzden ortak kısmı template metotta toplamak problemin bütününe hitap eder.
- Visitor kalıbı, hocanın deyişiyle "ağır sıklet" kalıplardandır ve eleştirisi de vardır. Kalıbın hedefi, bir nesne yapısının elemanları üzerinde ileride gerekecek yeni işlemleri, o elemanların sınıflarını değiştirmeden ekleyebilmektir. Hoca bunun kapalı kaynak koddan kaynaklanmadığını, proje sıfırdan yazılırken bile bu esnekliğin istendiğini vurguladı. Slaytın eleştirisine göre yapılacak işlemler önceden belli değilse sorumluluklar tam belirlenmemiş demektir ve sınıf yapılarını değiştirerek farklı bir tasarım yapmak daha doğru

olabilir. Ancak geriye doğru uyumluluğu çok uzun süre korunacak bir framework'te bütün ek işlevleri öngörmek mümkün olmadığından Visitor mantıklı hale gelir.

- Derleyici örneğinde `int x = 3+5` gibi bir komut düğümlerden oluşan bir ağaca dönüşür. Pretty printing, vurgulama, hata ayıklama, tür denetimi ve kod üretimi gibi işlerin her biri ağacı dolaşır ama her düğümde iş yapmaz; örneğin vurgulama yalnız tür anahtar kelimesini koyu yapar, değişken izleme yalnız değişken düğümlerine bakar. Bu işlerin hepsini Node hiyerarşisine metot olarak koymak, işin anlamsız olduğu düğümlerde de boş gövdeli metot yazmayı zorunlu kılar. Visitor çözümünde her iş ayrı bir somut ziyaretçi olur; düğümler yalnız `Accept` metodunu bilir ve ziyaretçinin kendi türlerine ait metodunu çağırır. Bunun bedeli, düğümlerin ziyaretçi arayüzündeki hangi metodun kendileri için olduğunu bilmesidir; hoca bunu küçük bir bağlaşım eleştirisi olarak dile getirdi. Yeni bir düğüm türü eklemek ise bütün ziyaretçilere yeni bir metot eklemeyi gerektirir.
- Proje yönetimi örneğinin ilk çözümü problemi çözer, ama her düğümde tür denetimi yapar: süre hesabında yalnız Task'lar, maliyet hesabında ise Deliverable'lar önemlidir ve bu ayrım `instanceof` ile yapılır. Hoca bunu "gözümü kapatırım, vazifemi yaparım" türünden hızlı bir çözüm olarak niteledi. İkinci çözümde her düğüm türü ziyaretçideki kendi metodunu çağırdığından tür denetimine gerek kalmaz; maliyet ve süre hesapları da ayrı sorumluluklar olarak ayrı sınıflara geçer. Ziyaretçinin kurucusu kök düğüme `accept(this)` diyerek dolaşmayı başlatır. Hoca, Visitor'ın getirdiği yararın bu kadar çabaya değip değmeyeceğinin projenin hedeflerine bağlı olduğunu, kalite hedeflerinin de amaç değil araç olması gerektiğini söyledi.
- Hoca, ders notu kaynak kodlarını Eclipse'e içe aktarırken uyumluluk sorunu çıkarsa yeni bir proje ve paket oluşturup örnek paketteki sınıfları bu pakete kopyalamanın bir çözüm olduğunu gösterdi.
- Mediator kalıbında amaç, birbiriyle konuşan çok sayıda nesnenin ilişkilerini tek bir aracı nesnede toplamaktır. Font penceresi örneğinde liste kutusu, giriş alanı ve düğmeler birbirine değil FontDialogDirector'a bağlanır; bir bileşen değişince `Changed()` ile director'a haber verir, director da hangi bileşenin değiştiğine bakıp gerekenleri sorgular ve önizlemeyi günceller. Hoca, bu yaklaşımın bağlantı sayısını azalttığını ama sınıf düzeyinde bağlaşımı değiştirmediğini, karmaşıklığı ConcreteMediator'a taşıdığını söyledi. Etkileşimler karmaşıksa kalıp yarar sağlar; değilse yalnız uzun ve karmaşık bir mediator sınıfı ortaya çıkar. Hoca bu kalıp için ayrı bir örnek gerçeklemeye gerek görmedi.

  Ders sonrası not: Hoca 5 Colleague arasındaki doğrudan bağlantıyı "5 kere 5, 25" diye saydı, sonra nasıl sayıldığından emin olmadığını belirtti (01:41:06). Bir nesnenin kendisiyle bağlantısı sayılmazsa 5 nesne arasında 10 yönsüz ($5 \times 4 = 20$ yönlü) bağlantı vardır; kalıbın "N^2" ifadesi bu sayının büyüme hızını anlatır. Mediator ile bağlantı sayısı 5'e iner.

- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.